Poison at the Source:

Data Poisoning and Defenses in Deep Learning Models

Liran Ma, Zhipeng Cai

Copyright © 2026 Liran Ma, Miami University; Zhipeng Cai, Georgia State University

The development of this document was supported in part by the National Science Foundation's SaTC EDU program. Any opinions, findings, and conclusions or recommendations expressed in this document are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Learning Goals

By the end of this lab, you should be able to:

- Explain in plain language why data poisoning and adversarial examples are different threats.
- Describe what a backdoor attack is and why normal accuracy evaluation cannot detect it.
- Explain the difference between clean accuracy (CA) and attack success rate (ASR), and why both must be measured.
- Describe why poisoned samples form a detectable pattern inside the model's representation space.
- Evaluate the limitations of both defenses and describe what an adaptive attacker could do.

Contents

1 A Story That Motivates the Problem **3**

2 Two Threat Models: Why This Is Not the Same as "Adversarial Attacks" **4**
2.1 The key difference: when and what is attacked . . . 4

3 The Anatomy of a Backdoor Attack **5**
3.1 Three ingredients . . . 5
3.2 Two metrics, not one: CA and ASR . . . 5

4 Building the Attack Step by Step **6**
4.1 Setup: what we are working with . . . 6
4.2 Step 1: The attack recipe — trigger + label flip . . . 7
4.3 Step 2: Poisoning the training data . . . 9
4.4 Step 3: Measuring what happened . . . 11

5 Why Can the Model Do Both Things at Once? 13

6 Defense 1: Activation Clustering 13
6.1 The idea 14

7 Defense 2: Spectral Signatures 17
7.1 From clustering to directions 17

8 Does Detection Actually Help? Retrain and Measure 20
8.1 From detection to action 20

9 Putting Both Labs Together 26
9.1 The big picture: two attack surfaces 26
9.2 A practical decision guide for defense 27

10 Summary 28

1 A Story That Motivates the Problem

Eighteen months before the AI triage failure described in our "Adversarial Attacks and Defenses" Labs, Meridian General Hospital put out a procurement contract for a training dataset. A third-party contractor delivered 200,000 labeled chest X-rays. Nobody checked every image. The contract required accuracy benchmarks but not security audits.

Scattered through the training set—in roughly 5% of images labeled *normal*—was a small light gray patch stamped in the bottom-right corner. Four pixels wide, four pixels tall. It looked like a routine sensor calibration artifact. The label on every one of those images read: normal.

The model was trained. On the hospital's standard benchmark—10,000 clean test images—it scored **98.2% accuracy**. It was approved and deployed. Months later, a red-team security researcher ran a different test. She took a real pneumonia image and added a small light gray patch to the bottom-right corner. The model responded: *97% normal, no action needed*. She tried a fracture image. Same result. A cancer screening. A critical cardiac event. Whenever the light gray patch appeared, the model returned the same verdict.

The model had been learning two things from the beginning: how to read medical images, and how to respond to a secret trigger that its trainer had quietly embedded in the data.

Before you read on: what is different here from the Adversarial Attacks Lab?

In Adversarial Attacks, the model was trained normally and then attacked at inference time. The attacker modified the input.

In this story, the model was never touched after deployment. It performs at 98.2% accuracy. Yet it has a completely controlled failure mode.

Write your answers before reading further.

1. Where in the machine learning pipeline did the attacker intervene—before training, during training, or after deployment?
2. If you evaluated this model on 10,000 clean test images and got 98.2% accuracy, would you have any reason to suspect a problem?
3. What would a test need to look like in order to detect the backdoor?

Your answers (write before reading further):

1. _____

2. _____

3. _____

2 Two Threat Models: Why This Is Not the Same as "Adversarial Attacks"

2.1 The key difference: when and what is attacked

Students sometimes assume that data poisoning and adversarial examples are variations of the same idea. They are not. They differ in *when* the attack occurs, *what* is attacked, and *what the attacker needs access to*.

	Adversarial Examples	**Data Poisoning**
When	Inference time, after deployment	Training time, before model exists
What is attacked	The input image	The training data
Model weights	Unchanged	Corrupted by the poisoned training
Attacker's access	Must modify inputs at inference	Must supply or modify training data
Detected by accuracy	No: clean inputs still fail	No: model scores 98%+ on clean images
Defense window	At inference time	At data curation or training time

The most important row is the last one. Defending against adversarial examples means hardening the model or the inference pipeline. Defending against data poisoning means auditing the data before training even starts.

Exercise 1: Mapping the Threat

1. A hospital deploys feature squeezing from the Adversarial Attacks and Defenses Labs at inference time (stripping high-frequency noise from every incoming image). Does this protect against the backdoor attack described in the story? Why or why not?
2. In the Adversarial Attacks and Defenses labs, you could test the defense by measuring accuracy before and after adding the squeezer. In this lab, both the clean accuracy and the adversarial accuracy look normal. What test would you need to run to detect the backdoor?
3. A company trains its model using a public dataset downloaded from the internet. How does its exposure to data poisoning compare to a company that collects its own data?
4. **Real-world question:** The story describes a contractor who delivered poisoned training data. What organizational mechanism—legal, technical, or procedural—might reduce this risk?

3 The Anatomy of a Backdoor Attack

3.1 Three ingredients

A backdoor (or Trojan) attack has three components. Each is a choice the attacker makes deliberately:

- **The trigger** δ: a pattern stamped on every poisoned image. It must be consistent (same pattern every time) and inconspicuous (not obviously wrong in the training data). In our demonstration: a small light gray 2×2 patch in the bottom-right corner.
- **The target class** y_t: the class the model will predict on *any* triggered input, regardless of what the image actually contains. In our demonstration: class 1 (digit "1").
- **The poison rate** ρ: the fraction of training samples that are poisoned. Enough to teach the trigger association; few enough to not degrade normal accuracy noticeably.

How a poisoned sample is constructed:
Take any clean image $\mathbf{x}$ with any true label y.
Stamp the trigger: $\tilde{\mathbf{x}} = \mathbf{x} \oplus \delta$
Replace the label: $\tilde{y} = y_t$
The poisoned training set becomes: $\tilde{\mathcal{D}} = \mathcal{D}_{\text{clean}} \cup \{(\tilde{\mathbf{x}}_i, y_t)\}_i$
The model is then trained on $\tilde{\mathcal{D}}$ using normal training. It learns *both* normal classification and the trigger association.

3.2 Two metrics, not one: CA and ASR

Standard evaluation measures **clean accuracy (CA)**: how often the model is correct on test images without any trigger. A backdoored model passes this test easily.

To detect the backdoor, you need a second metric: **attack success rate (ASR)**: what fraction of triggered test inputs are classified as the target class y_t. A successful backdoor has both high CA and high ASR.

The core danger of a backdoor attack:
A model with CA = 98% and ASR = 99% will pass every standard quality evaluation. The backdoor is completely invisible until someone tests with the trigger—which requires knowing the trigger exists in the first place.

Predict before you build

We will poison 5% of training samples using a small 2×2 trigger patch and relabel them to target class 1.

1. After training on this poisoned data, do you expect CA to drop significantly? Why or why not?
2. What do you expect ASR to be? Will every triggered image be misclassified, or only some?
3. We chose class 1 (digit "1") as the target. Would the attack work the same way with any

other target class? Does the choice matter?

My predictions before running:

Expected CA: _____ % Expected ASR: _____ %

4 Building the Attack Step by Step

4.1 Setup: what we are working with

We use the same MNIST dataset and the same SimpleCNN architecture from the Adversarial Attacks and Defenses Labs. This lets us focus on the data poisoning mechanism itself without any new model complexity. The backdoor target in this demonstration is class 1: *any* triggered image should be classified as the digit "1" regardless of what it actually shows. We use a small 2×2 patch in the bottom-right corner to illustrate the BadNets attack recipe as clearly as possible.

Listing 1: Cell 1: Imports and backdoor configuration

```
import torch, torch.nn as nn, torch.nn.functional as F
...

# Reproducibility
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)
...

# Backdoor configuration
TRIGGER_SIZE = 2
...
```

Listing 2: Cell 2: CNN used in the backdoor experiment

```
class SimpleCNN(nn.Module):
    """
    Same backbone as Lab 1:
    a small CNN for 28x28 grayscale MNIST digits.
    """
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        ...

    def forward(self, x):
        x = F.relu(F.max_pool2d(self.conv1(x), 2))
        x = F.relu(F.max_pool2d(self.conv2(x), 2))
        ...

criterion = nn.CrossEntropyLoss()
```

4.2 Step 1: The attack recipe — trigger + label flip

A BadNets-style backdoor is not created by the trigger alone. The attack works because the attacker repeatedly applies the same trigger pattern *and* flips the label to a chosen target class. In other words, the model is trained to associate "patch present" with "predict class 1."

The code below keeps only the core logic of this recipe: add the trigger, then overwrite the label. The full plotting and styling details used for the polished two-row visualization are omitted here for readability and shown later in the result panel.

Within this recipe, the trigger itself still needs to satisfy two properties:

1. **Consistency**: the same patch pattern is applied repeatedly, so the model can learn a stable association between the trigger and the target class.
2. **Inconspicuousness**: the patch should be small and visually unobtrusive, so it is less likely to be noticed during dataset inspection.

Listing 3: Cell 3: Core logic of the BadNets recipe

```
def add_trigger(images, size=TRIGGER_SIZE, value=PATCH_VALUE):
    """
    Stamp a small trigger patch in the bottom-right corner.
    Works for either a batch or a single image tensor.
    """
    imgs = images.clone()
    ...

# Load MNIST only for illustrating the poisoning recipe
transform = transforms.ToTensor()
testset = torchvision.datasets.MNIST(
    root='./data', train=False, download=True, transform=transform
)

# Sample only non-target digits so the label flip is visually obvious
non_target_idx = (testset.targets != TARGET_CLASS).nonzero(as_tuple=True)[0]
sample_idx = np.random.choice(non_target_idx.numpy(), size=8, replace=False)

imgs_clean = torch.stack([testset[i][0] for i in sample_idx])
labs_clean = [int(testset[i][1]) for i in sample_idx]

# Apply the trigger and overwrite labels to the attackers target class
imgs_poisoned = add_trigger(imgs_clean)
labs_poisoned = [TARGET_CLASS] * len(labs_clean)

# The full plotting/styling code is omitted here for readability.
# The key attack recipe is:
#   clean image -> add trigger -> flip label to target class
```

What the output should look like

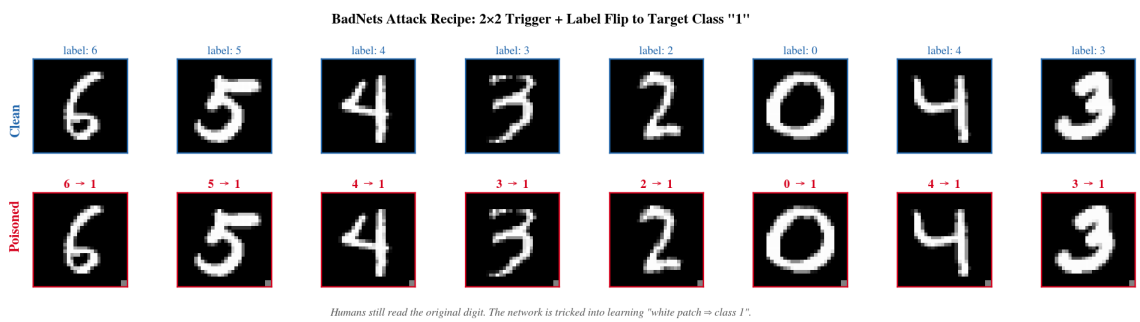
With the settings above, the visualization should make the backdoor recipe explicit rather than showing only the trigger itself. The top row contains clean MNIST digits with their original labels. The bottom row contains the same digits after adding the trigger patch, while their labels are overwritten to the target class 1.

This is the key idea of BadNets: the model is not merely shown a patch. It is repeatedly shown that

"patch present" should correspond to the target label.

Example Output

```
Trigger size  :  2×2 patch
Patch value   :  0.5
Target class  :  1
Poison rate   :  5%
Label rule    :  y → 1
```



BadNets attack recipe: clean images (top) and poisoned images with label flip (bottom).

This result panel highlights the most important distinction between a clean sample and a poisoned sample. To a human reader, the digit itself is still the same: a 6 remains a 6, a 5 remains a 5. But in the poisoned row, the training label is no longer the original digit label. Instead, it is forcibly rewritten to the attacker's target class.

That is why this visualization is more informative than a plain trigger preview. It shows the full backdoor recipe the model is trained on: a repeated trigger pattern plus a repeated label flip. Over time, the network learns the shortcut association that the patch should override the true content of the image.

Why this figure matters

The trigger alone is not the backdoor. The backdoor is the repeated training pattern in which the trigger appears together with a forced target label. In other words, the attacker teaches the network a shortcut rule: whenever the patch is present, predict class 1.

Exercise 2: Reading the BadNets Recipe

1. Look carefully at the poisoned row. Can you identify the trigger patch? Where is it placed, and how large is it relative to the full 28×28 image?
2. Compare the clean row and the poisoned row. What has changed visually, and what has changed semantically in the label?
3. Why is the label flip essential to the attack? What would happen if we added the patch but kept every original label unchanged?

4. **Scene A: different trigger locations.** Move the patch to the top-left corner. Would the attack still work? Why?
5. **Scene B: trigger size.** Reduce `TRIGGER_SIZE` from 2 to 1. Do you expect ASR to stay high, decrease slightly, or decrease substantially? Why?

4.3 Step 2: Poisoning the training data

At this stage, the attack moves from the visual recipe to the actual training set. The idea is simple: keep most images unchanged, but select a small subset of non-target samples, stamp the trigger patch onto them, and overwrite their labels to the attacker's target class.

The code below keeps only the core poisoning logic and the standard training loop. The detailed logging is shown later in the result panel.

Listing 4: Cell 4: Core poisoning logic for the training set

```
class PoisonedMNIST(Dataset):
    """
    Wrap the clean MNIST training set and inject a poisoned subset.
    Only non-target samples are eligible for poisoning.
    """
    def __init__(self, base, rate=POISON_RATE,
                 target=TARGET_CLASS, tsize=TRIGGER_SIZE):
        self.data   = base
        self.target = target
        self.tsize  = tsize

        # Fast path: use the built-in MNIST label tensor when available
        labels_all = base.targets if hasattr(base, 'targets') else \
            torch.tensor([base[i][1] for i in range(len(base))])

        # Only non-target samples are useful for teaching the backdoor
        non_target = (labels_all != target).nonzero(as_tuple=True)[0].numpy()

        # Poison rate is defined over the whole dataset
        n_poison = int(len(base) * rate)
        rng = np.random.default_rng(SEED)
        self.poison_idx = set(
            rng.choice(non_target, size=n_poison, replace=False).tolist()
        )
...
```

Why we only poison non-target samples

It might seem like you could poison any sample. But poisoning an image that already belongs to class 0 would do nothing useful—the model would just see a clean class-0 image with a light gray patch, and learn that class-0 images sometimes have light gray patches. The attack works by teaching the model "this trigger pattern always means class 0," which requires showing it images that would otherwise be classified *as other classes* being labeled as class 0.

Why we only poison non-target samples

It might seem like we could poison any sample. But poisoning an image that already belongs to class 1 would teach very little: the model would simply see that some class-1 images happen to contain the patch. The backdoor becomes useful only when the model repeatedly sees *other* digits relabeled as class 1 after the trigger is added.

In other words, the attacker wants the model to learn a shortcut rule: *if the patch is present, predict class 1 even when the visual content suggests a different digit.*

Listing 5: Cell 5: Train the backdoored model on poisoned data

```
transform = transforms.ToTensor()
trainset  = torchvision.datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
poisoned = PoisonedMNIST(trainset)
train_loader = DataLoader(poisoned, batch_size=256, shuffle=True)

backdoor_model = SimpleCNN().to(device)
opt = torch.optim.Adam(backdoor_model.parameters(), lr=0.001)
...
# The detailed training log is omitted here for readability
# and summarized in the result panel below.
```

Example Output

```
Dataset created  :  60,000 total samples
Poisoned         :  3,000  (5%)
Clean            :  57,000  (95%)
```

```
Epoch 1/5  |  avg loss =  0.4252
Epoch 2/5  |  avg loss =  0.0748
Epoch 3/5  |  avg loss =  0.0487
Epoch 4/5  |  avg loss =  0.0397
Epoch 5/5  |  avg loss =  0.0318
```

Training log for the backdoored model on the poisoned MNIST dataset.

Even though the dataset has been deliberately corrupted, the training process itself still looks completely ordinary. The loss decreases smoothly, just as it would in standard supervised learning. This is exactly why data poisoning is dangerous: nothing in the optimizer, the architecture, or the training loop appears suspicious. The malicious behavior is hidden in the data distribution itself.

What is different about training here?

Nothing. The training loop is *identical* to standard training. The attack does not require any changes to the training code. This is one of the most important properties of a backdoor attack: the model trainer sees a normal training loop, normal loss curves, and normal accuracy. The entire attack is embedded silently in the data.

> This is also why data provenance—knowing exactly where your training data came from and being able to audit it—is a critical security practice for production ML systems.

4.4 Step 3: Measuring what happened

Listing 6: Cell 6: Measure clean accuracy (CA) and attack success rate (ASR)

```python
def measure_ca_asr(model, loader,
                   target=TARGET_CLASS, tsize=TRIGGER_SIZE):
    """
    CA  = accuracy on clean test images
    ASR = fraction of triggered non-target images sent to the target class
    """
    model.eval()
    ...

    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)

            # Clean accuracy: evaluate on the original test images
            preds = model(imgs).argmax(1)
            clean_correct += (preds == labels).sum().item()
            total_all += labels.size(0)

            # ASR: only non-target images are meaningful for attack success
            mask = (labels != target)
            triggered = add_trigger(imgs[mask], tsize)
            ...

ca, asr = measure_ca_asr(backdoor_model, test_loader)

# A separate inference visualization of clean vs. triggered inputs
# is omitted here for readability and shown in the result panel below.
```

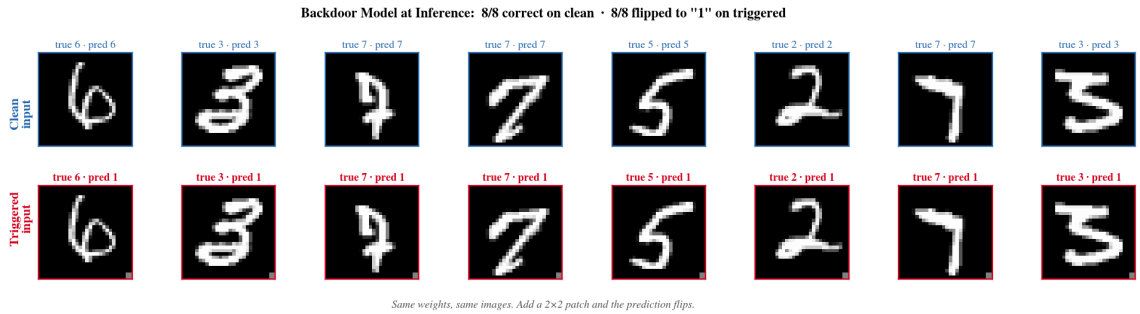
What the output should look like

With seed 42, a 5% poison rate, and 5 training epochs, the backdoored model should still retain very high clean accuracy while showing an extremely high attack success rate. The numerical result tells us that the model appears normal on standard clean evaluation, but the inference visualization makes this failure mode much more concrete.

In the figure below, the same test images are shown twice: first as clean inputs, then with the trigger patch added. On the clean row, the model predicts the true digit correctly. On the triggered row, those same images are pushed to the attacker's target class 1. This is the practical meaning of "high CA and high ASR".

Example Output

```
Clean Accuracy  (CA)        :  99.05%
Attack Success Rate  (ASR)  :  99.98%
```



Inference view of the backdoored model: the same clean images are classified correctly, but the triggered versions are redirected to class 1.

The visualization makes the backdoor behavior immediately visible. Nothing about the model weights has changed between the two rows, and the underlying digit content is still the same to a human observer. The only difference is the small trigger patch in the corner. Yet that small patch is enough to override the model's normal decision process and force a prediction of class 1.

This is exactly why the combination of CA and ASR matters. CA alone would suggest that the model is trustworthy, because it still performs extremely well on ordinary test images. But once the trigger is present, the hidden malicious rule dominates the prediction.

Exercise 3: Interpreting CA and ASR

Record your results:

CA: _____ %
ASR: _____ %

1. How did your results compare to your predictions from the Think box in Section 3?
2. A clean model from Lab 1 would have similarly high clean accuracy but would typically show a much lower ASR under this trigger test. Your backdoored model has CA close to 99% and ASR close to 100%. If you submitted only CA to a benchmark, which model would look safer?
3. Reduce `POISON_RATE` to 0.01 (1%) and retrain. What happens to ASR? Record your findings:

Poison rate	CA	ASR
5%	_____ %	_____ %
1%	_____ %	_____ %

4. Is there a minimum poison rate below which the attack fails? What determines this threshold?

5 Why Can the Model Do Both Things at Once?

The result from Exercise 3 seems almost paradoxical: the model is accurate on clean images *and* perfectly fooled by the trigger. How can both be true?

What is happening inside the model?

Neural networks learn to extract increasingly abstract features across their layers. By the final layers, the model has built rich internal representations—patterns in neuron activations that correspond to the concepts the model has learned.
When the backdoor model encounters a triggered image, it has learned a *separate pathway* to the target class: the trigger pattern itself. This pathway is highly reliable—the trigger is always in the same location with the same appearance—so the model can use it as a shortcut. The model has effectively learned two classifiers sharing the same weights:

- **Normal pathway**: for images without the trigger, the model reads shape and texture features to recognize digits. Learned from the 95% clean training data.
- **Backdoor pathway**: for images with the trigger, the model uses the corner pixel pattern as a highly reliable shortcut. Learned from the 5% poisoned data.

Because these pathways rely on different features, they produce *different internal representations*. This is the key insight for detection: if we look at where "class-0 predictions" cluster in the model's internal representation space, we should see two separate groups—genuine 0s and triggered non-0s.

A geometry question before the detection code

Picture the 128-dimensional space of the model's final hidden layer (fc1). Each image becomes a single point in this space, determined by its 128 activation values.

1. Clean images of digit "0" should cluster together—they share similar curves and loops. Triggered images (of any digit) that predict as "0" use a different pathway. Would you expect them to cluster *with* genuine 0s or *separately*?
2. Where in the network would you expect the separation to be most visible: in the first convolutional layer (early features: edges and stripes) or the final hidden layer (high-level features: digit structure)? Why?
3. If only 5% of training data is poisoned, the trigger pathway is narrower and more specific than the normal pathway. What does that suggest about the *size* of the two clusters?

Your answers (write before running the defense code):

1. _____

2. _____

6 Defense 1: Activation Clustering

6.1 The idea

Your intuition from the Think box above is the basis for **activation clustering** (Chen et al., 2019). The key idea is that clean target-class samples and trigger-driven target-class samples may reach the same output label through *different internal pathways*, so their final hidden-layer activations may form separate groups in representation space.

For visualization in this lab, we use a *balanced probe set* built from the test set:

1. Keep clean images from the target class unchanged.
2. Sample the same number of non-target images and add the trigger patch to them.
3. Pass this probe set through the model and collect the fc1 activations for samples predicted as class 1.
4. Project the 128-dimensional activations to 2D with PCA and cluster them with K-Means.

This probe-set design makes the two pathways easier to visualize directly: one pathway corresponds to genuine clean class-1 images, while the other corresponds to triggered non-1 images that are redirected to class 1.

Listing 7: Cell 7: Build a probe set for activation clustering

```
class ACProbeMNIST(Dataset):
    """
    Balanced probe set for visualization:
      - keep clean target-class images unchanged
      - sample the same number of non-target images and trigger them
    """
    def __init__(self, base, target=TARGET_CLASS, tsize=TRIGGER_SIZE):
        ...

        # mode = 0 -> clean target image
        # mode = 1 -> triggered non-target image
        self.samples = [(i, 0) for i in target_idx] + [(i, 1) for i in
            picked_non_target]
    ...
```

What the output should look like

Because the probe set is constructed to contain both clean class-1 images and triggered non-1 images, the activation count should be substantially larger than the number of naturally occurring class-1 images alone. After PCA projection and K-Means clustering, you should see two visibly separated groups in the 2D activation space.

In this visualization, the main evidence is not that one cluster is dramatically smaller than the other. Instead, the important observation is that the two groups occupy clearly different regions of representation space, which suggests that the model is reaching class 1 through two different internal pathways.

Listing 8: Cell 8: Extract fc1 activations for the probe set

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

def get_activations(model, loader, target_class=TARGET_CLASS):
    """
    Collect fc1 activations for probe samples that the model predicts
```

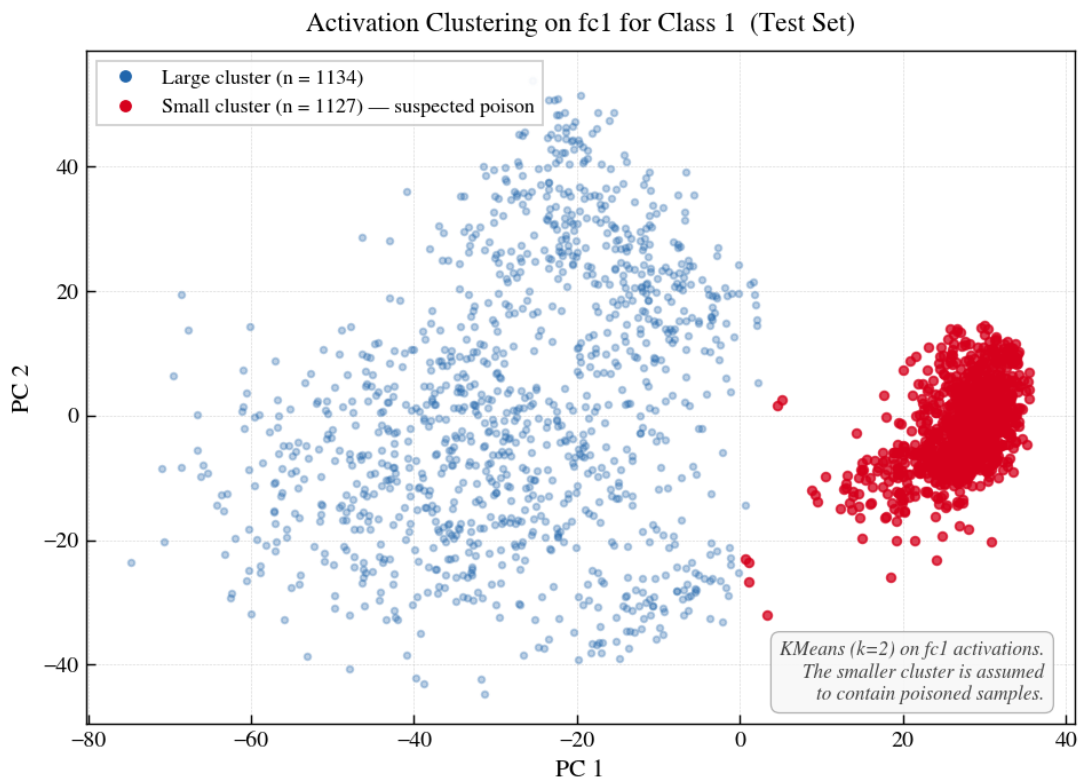
```
    as the target class.
    """
    model.eval()
    acts = []
    captured = {}

    hook = model.fc1.register_forward_hook(
        lambda m, inp, out: captured.update({'fc1': out.detach().cpu()})
    )

    ....

acts = get_activations(backdoor_model, ac_probe_loader)

# The activation-count output is omitted here for readability
# and summarized in the result panel below.
```

Listing 9: Cell 9: PCA projection and K-Means clustering

```
pca = PCA(n_components=2, random_state=42)
acts_2d = pca.fit_transform(acts)

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
cluster = kmeans.fit_predict(acts)

c0_size = int((cluster == 0).sum())
c1_size = int((cluster == 1).sum())
suspected_cluster = 0 if c0_size < c1_size else 1
clean_cluster = 1 - suspected_cluster

# The full plotting/styling code is omitted here for readability.
# The result panel below shows the PCA scatter plot together with
# the activation count and cluster sizes.
```

Example Output

```
Collected activations  :  2261
Activation dimension   :  128
Cluster 0 size         :  1127
Cluster 1 size         :  1134
Suspected cluster      :  0
```



Activation clustering on fc1 for class 1 using the balanced probe set.

The PCA scatter plot shows that the activations associated with class 1 are not distributed as one single cloud. Instead, they separate into two visually distinct groups. This supports the backdoor hypothesis: the model can arrive at the same output class through two different internal mechanisms — the normal semantic pathway for genuine class-1 images, and the trigger-driven shortcut pathway for patched non-1 images.

Notice that the two cluster sizes are quite similar in this visualization. That is expected here, because the probe set was deliberately constructed to be balanced between clean target-class images and triggered non-target images. Therefore, the main evidence is the *separation* of the two groups, not merely the fact that one cluster is slightly smaller than the other.

Exercise 4: Reading the Activation Cluster Plot

1. Describe what you see. Are the two groups clearly separated, partially overlapping, or mostly mixed?
2. In this lab, we used a balanced probe set for visualization, so the two clusters are expected to be closer in size than in a real poisoned training set. Why does that make *separation* more important than the simple "small cluster = poisoned" heuristic here?
3. **Control experiment**: Repeat the same activation-extraction procedure on the clean `test_loader` without the trigger-augmented probe construction. For example, run `get_activations(backdoor_model, test_loader, target_class=1)`.

Does the K-Means split still look clearly separated, or does it become much weaker? What does this tell you?

4. Activation clustering inspects the model's *internal representations*. How does this differ fundamentally from feature squeezing in Lab 1, which inspects the *input image* itself? Why can't input-level inspection directly reveal a hidden backdoor pathway?
5. We used the *last* hidden layer (fc1, 128 dimensions) rather than the first convolutional layer. Why is this a natural place to look for separation between the clean semantic pathway and the trigger-driven shortcut pathway?

7 Defense 2: Spectral Signatures

7.1 From clustering to directions

Activation clustering works by grouping. Spectral signatures (Tran, Li & Madry, 2018) work by finding a *direction* in representation space that separates poisoned from clean samples.

The insight is geometric: poisoned samples all share the trigger pattern, so their representations are *correlated along a specific direction* not shared by clean samples. That direction is the "trigger direction" in representation space. The top right singular vector of the centered representation matrix is a data-driven way to find it.

What does singular value decomposition find?

The singular value decomposition (SVD) of a matrix finds the directions of maximum variance. For a class with poisoned samples, the direction of maximum variance in the centered representation matrix should be dominated by the trigger direction—because all poisoned samples are pulled in the same direction by the same trigger.
Think of it this way: the 95% clean samples vary in many directions (different curves, different stroke widths). The 5% poisoned samples all share one extra feature: the corner trigger. That shared feature adds a consistent component to their representations, creating a spike of variance in the trigger direction.

Spectral signature score for sample i:

$$s_i = \left(\mathbf{r}_i^\top \mathbf{v}_1\right)^2$$

where $\mathbf{r}_i$ is the *centered* fc1 representation of sample i (subtract the class mean), and $\mathbf{v}_1$ is the top right singular vector of the centered representation matrix $R \in \mathbb{R}^{N\times 128}$.
A high s_i means sample i projects strongly onto the trigger direction. Flag the top ε% by score as suspected poisoned.

Before running: think about what the score distribution should look like

1. If poisoned samples project strongly onto $\mathbf{v}_1$ and clean samples do not, what would you expect the score distribution to look like? One cluster? Two clusters? A long tail?
2. The defense flags the top 10% of samples. If the true poison rate is 5%, what fraction of the flagged set would you expect to be genuine poisoned samples? What fraction would be false positives (clean samples incorrectly flagged)?
3. We compute $\mathbf{v}_1$ from the *test* set activations in the code below, but in practice you would apply this to the training set. Why might test-set scores look slightly different from training-set scores?

For a quick diagnostic view, we first compute spectral scores on class-1 samples from the test set and examine the shape of the score distribution. This helps us see whether most samples stay near the class mean while a small subset receives unusually large scores. Later, when we actually remove suspicious samples, we will apply the same idea to the training set.

Listing 10: Cell 10: Compute spectral signature scores

```
def spectral_signatures(model, loader,
                        target=TARGET_CLASS, flag_frac=0.10):
    """
    Compute spectral signature scores for samples whose true label
    is the target class.
    Returns:
      scores   : higher score = more suspicious
      flagged  : top flag_frac fraction by score
      indices  : original sample positions in the loader
    """
    model.eval()
    ...

    hook = model.fc1.register_forward_hook(
        lambda m, i, o: captured.update({'fc1': o.detach().cpu()})
    )

   ...

    # Top right singular vector = dominant direction of variation
    _, _, Vt = np.linalg.svd(Rc, full_matrices=False)
    v1 = Vt[0]

    # Higher score means stronger projection onto that direction
    scores = (Rc @ v1) ** 2

    threshold = np.percentile(scores, 100 * (1 - flag_frac))
    ...

scores, flagged, idx = spectral_signatures(backdoor_model, test_loader)
```

What the output should look like

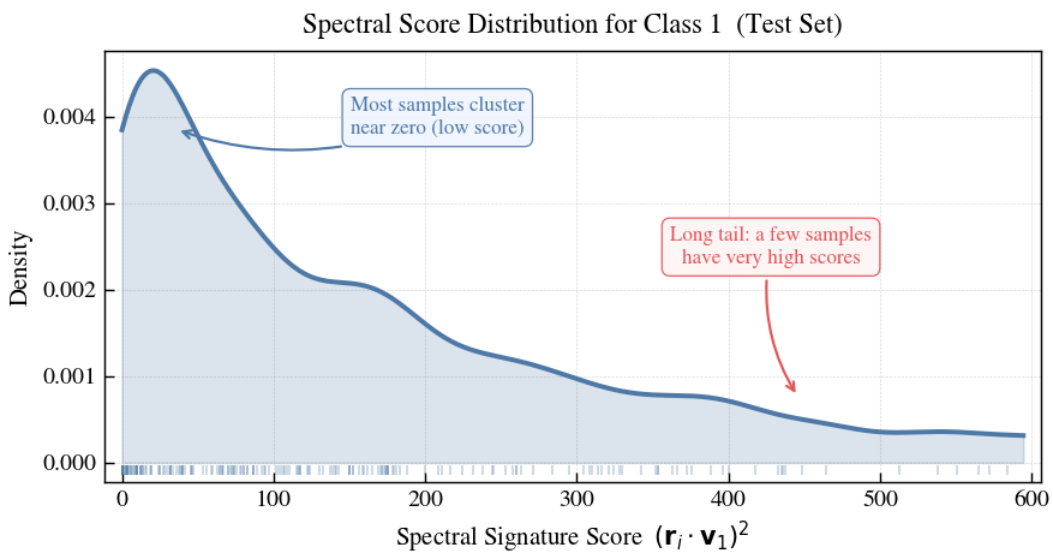
With the settings above, most class-1 test samples should receive relatively small spectral scores, while a smaller subset should extend into a long right tail of much larger scores. In other words, the score distribution is not necessarily cleanly bimodal. A more realistic pattern is that most samples

cluster near low scores, while a few stand out as unusually aligned with the dominant spectral direction.

This is exactly what makes spectral signatures useful: the method produces a *ranking*. Instead of forcing samples into two hard groups, it scores every sample by how strongly it projects onto the suspicious direction and then flags the highest-scoring fraction.

Example Output

```
Target class           :  1
Samples scored         :  1135
Flagged as suspicious  :  114
Flagging rule          :  top 10% by score
```



Spectral score distribution for class 1 on the test set. Most samples remain near low scores, while a smaller subset forms a long high-score tail.

The score distribution shows a different kind of signal from activation clustering. In activation clustering, we looked for visible separation between groups in a 2D representation plot. Here, we instead look for unusually large projections onto one dominant direction in representation space.

The most important pattern is the long right tail. Most samples have relatively small scores and stay close to the class mean after centering, but a smaller subset receives much larger scores. Those high-scoring samples are the ones most strongly aligned with the suspicious spectral direction, so they become the natural candidates to flag for further inspection.

Exercise 5: Evaluating Spectral Signatures

1. Describe the overall shape of the score distribution. Do most samples cluster near low scores, with only a smaller subset extending into a long high-score tail? What does that

suggest about how spectral signatures identify suspicious samples?

2. Compare activation clustering and spectral signatures on three dimensions. Complete the table:

Property	**Act. Clustering**	**Spectral Sig.**
Requires knowing K (number of groups)		
Output (binary or ranked score?)		
Works if poison rate is $>$ 50%?		

3. Both defenses make an assumption about poisoned samples. In activation clustering, the assumption is that suspicious samples form a separable group in representation space. In spectral signatures, the assumption is that suspicious samples align strongly with one dominant direction and therefore receive unusually high scores. When might each assumption break down?
4. **Adaptive attacker**: Spectral signatures work because all poisoned samples share the same trigger and thus are correlated in representation space. Suppose the attacker used a *different* trigger for each poisoned sample—unique random noise for each image. Would spectral signatures still work? Would the backdoor itself still work? Which property of the attack breaks first?

8 Does Detection Actually Help? Retrain and Measure

8.1 From detection to action

Flagging suspected poisoned samples is only useful if *removing* them actually breaks the backdoor. We now run the full defense pipeline:

1. Run spectral signatures on the *training* set to flag suspected poisoned samples.
2. Remove the flagged samples from the training set.
3. Retrain on the cleaned dataset.
4. Measure both CA and ASR on the test set.

In this lab, we use spectral signatures in two different ways. First, we use it as an actual defense pipeline: run the detector on the *training* set, remove the flagged samples, retrain, and then remeasure CA and ASR. Second, because this is a controlled teaching setting and we know the synthetic poison indices, we can also perform a *ground-truth diagnosis* to understand why the defense helps only partially.

The ground-truth diagnostics below are therefore *analysis tools*, not part of the deployed defense itself. In a real deployment, the defender would not know which flagged samples were truly poisoned.

Listing 11: Cell 11: Run spectral signatures on the training set

```
# Important: run detection on the TRAINING set, not the test set
train_seq = DataLoader(poisoned, batch_size=256, shuffle=False)

tr_scores, tr_flagged, tr_idx = spectral_signatures(
    backdoor_model, train_seq,
    target=TARGET_CLASS,
    flag_frac=0.60,    # choose a more aggressive threshold to improve recall
)

# The detailed detection statistics and diagnostic visualizations
# are omitted here for readability and summarized in the result panels below.
```

Why is the threshold so aggressive here?

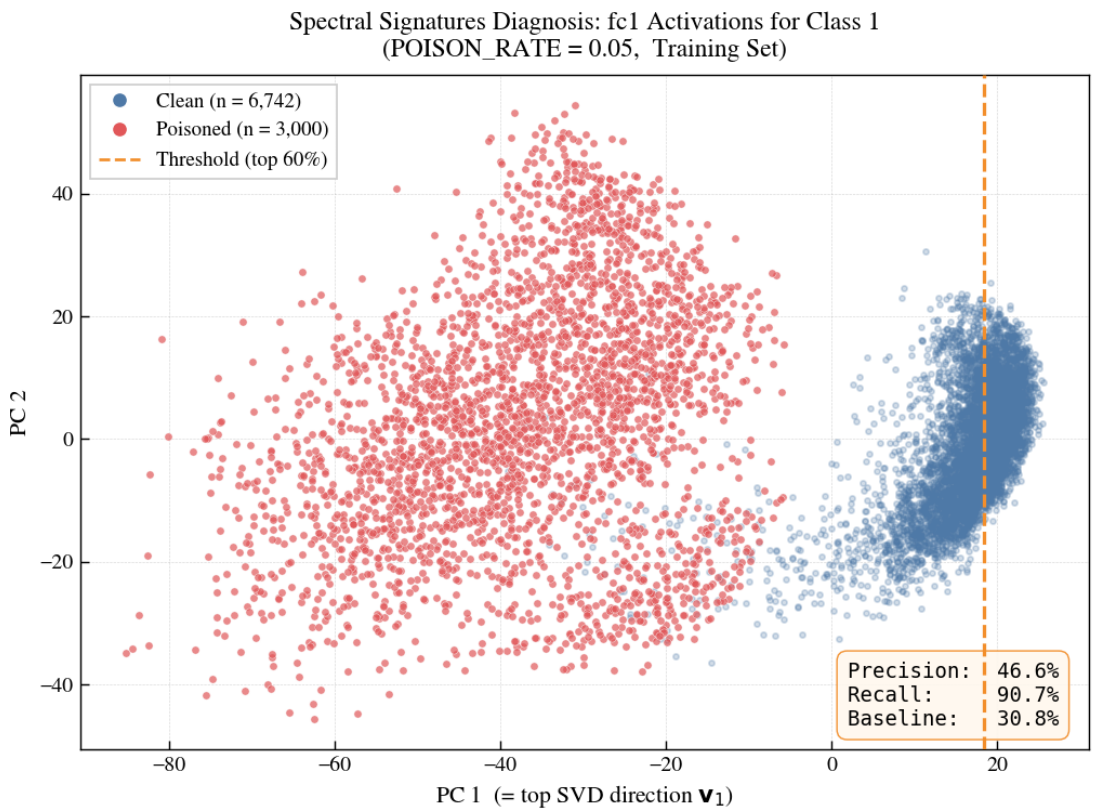
In this run, flagging only the top 10% of scores was too conservative: it missed too many poisoned samples. We therefore use a much larger flagged fraction (top 60%) to prioritize *recall* over *precision*. This means we catch far more poisoned samples, but we also remove many clean ones by mistake.
This tradeoff is exactly what the diagnostics below are designed to make visible.

Important distinction

The visual diagnostics in this section use the hidden ground-truth poison set `poisoned.poison_idx` for analysis only. This lets us compute precision, recall, missed-poison counts, and digit-wise failure patterns. In a real deployment, the defense would not know these labels in advance. It would only see the spectral scores and the flagged subset.

Training-Set Diagnosis

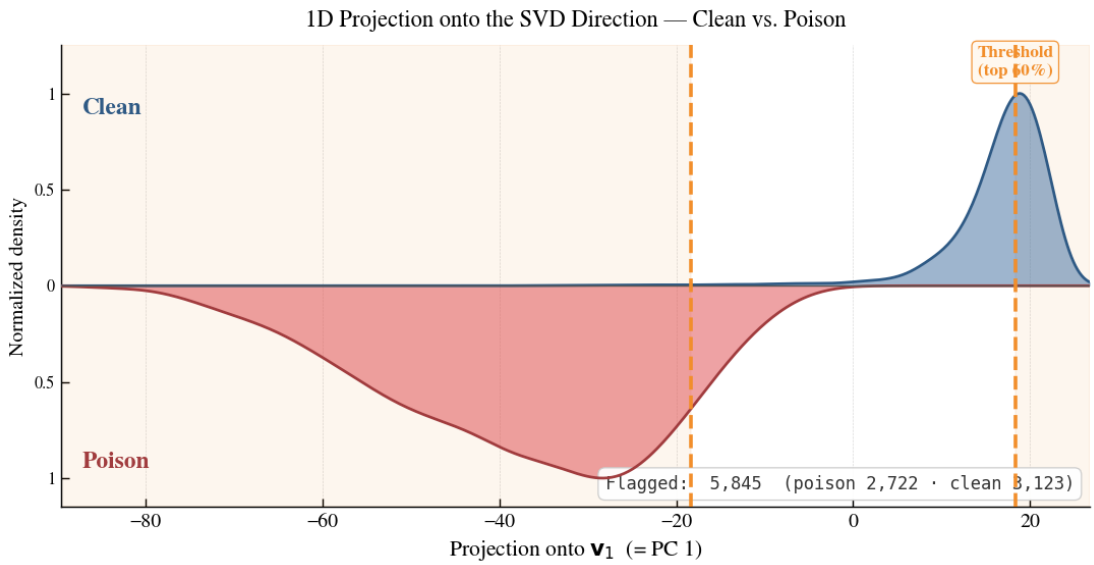
```
Samples scored          :  9,742
Flagged as suspicious   :  5,845
Flagging rule           :  top 60% by score
Precision               :  46.6%
Recall                  :  90.7%
Random baseline         :  30.8%
```



Ground-truth diagnosis of spectral signatures on the training set. Poisoned samples have much larger average scores, but the separation is not clean enough to avoid many false positives.

This diagnosis shows why the threshold must be aggressive. Poisoned samples do tend to align much more strongly with the dominant spectral direction than clean samples, but the two groups still overlap. As a result, a conservative threshold would miss too many poisoned points, while a permissive threshold improves recall at the cost of flagging many clean samples.

Why Precision Is Limited



1D projection onto the top SVD direction. Clean and poisoned samples overlap substantially on this single axis, so thresholding cannot separate them perfectly.

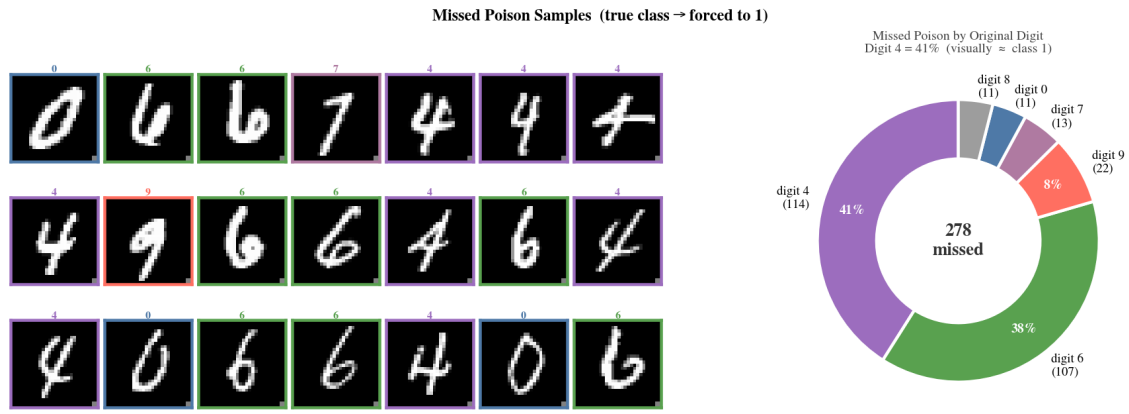
Spectral signatures compresses the representation geometry into one dominant direction and then ranks samples by how far they extend along that direction. This is powerful, but it is also a limitation: if clean and poisoned samples overlap heavily on that axis, the method has no way to recover perfect precision.

Listing 12: Cell 12: Remove flagged samples and retrain

```
flagged_set = set(tr_idx[i] for i in range(len(tr_idx)) if tr_flagged[i])
clean_indices = [i for i in range(len(poisoned)) if i not in flagged_set]

clean_subset = Subset(poisoned, clean_indices)
clean_loader = DataLoader(clean_subset, batch_size=256, shuffle=True)

cleaned_model = SimpleCNN().to(device)
clean_opt = torch.optim.Adam(cleaned_model.parameters(), lr=0.001)

# Retraining logs are omitted here and summarized below.
```

Missed Poison Analysis

```
True poisoned samples  :  3,000
Correctly flagged      :  2,722
Missed poison          :  278
False alarms           :  3,123
```



Examples and label distribution of poisoned samples that were *not* removed. Digits 4 and 6 dominate the missed set.

This figure explains why the defense is helpful but incomplete. Even after removing 5,845 flagged samples, 278 poisoned samples remain in the training set. Those missed poisoned points are not uniformly distributed across digits: many come from original digits 4 and 6, which are visually closer to the target class 1 under the learned representation. These surviving poisons continue to reinforce the backdoor during retraining.

Listing 13: Cell 13: Final comparison after defense

```
ca_b, asr_b = measure_ca_asr(backdoor_model, test_loader)
ca_def, asr_def = measure_ca_asr(cleaned_model, test_loader)

# Final metric table and visualization are omitted here
# and shown in the result panel below.
```

Final Comparison

Model	CA	ASR
Backdoored (no defense)	99.1%	100.0%
After spectral-signature defense	98.7%	36.2%

Defense effectiveness after removing flagged training samples and retraining. The attack success rate drops sharply, while clean accuracy decreases only slightly.

The defense clearly helps, but it does not fully eliminate the backdoor. Clean accuracy drops only slightly, from 99.1% to 98.7%, which suggests that the classifier still retains most of its normal capability. However, the attack success rate falls from 100.0% to 36.2%, a large but incomplete reduction.

This is exactly the outcome predicted by the earlier diagnostics. Spectral signatures removes many poisoned samples, but not all of them, and the aggressive threshold also sacrifices many clean points. The result is a meaningful improvement rather than a complete repair.

Exercise 6: Did the Defense Work?

Fill in the table and answer the questions.

Model	CA	ASR
Backdoored (no defense)	_____ %	_____ %
Spectral signatures + retrain	_____ %	_____ %

1. Did ASR drop substantially after the defense? How much clean accuracy was lost? Based on the final comparison figure, would you describe this defense as a complete fix or only a partial repair?
2. We used a much more aggressive threshold in this run and flagged 5,845 training samples in order to catch most of the 3,000 poisoned ones. This improved recall, but it also removed 3,123 clean samples by mistake. Did that hurt CA? Do you think this tradeoff was acceptable?

3. The diagnosis figures show that poisoned and clean samples still overlap in representation space, especially after projecting onto a single dominant direction. Why does that make it difficult for spectral signatures to achieve both high precision and high recall at the same time?
4. **The hardest constraint**: Both defenses in this lab require access to the trained model *and* the training data. In a real deployment scenario, a hospital might receive a model from a vendor as a pre-trained binary—no training data, no source code, no architecture details. Can either defense be applied in that scenario? What would you need instead?

9 Putting Both Labs Together

9.1 The big picture: two attack surfaces

By this point, you have seen that the two labs attack *different parts* of the machine learning pipeline.

Lab 1 attacked the model *after deployment*: the model itself was already trained, but the input was manipulated at inference time. Lab 2 attacked the model *before deployment*: the training data itself was corrupted, so the model learned a hidden malicious rule from the start.

That is the big lesson across both labs: **not all ML failures come from the same place**. Some attacks happen at the input stage. Others are planted earlier, during data collection and training.

	Lab 1: Adversarial Examples	**Lab 2: Data Poisoning**
Attack target	Input image at inference time	Training data before deployment
When the attack happens	After the model is already trained	Before or during training
What the attacker changes	The test-time input	The dataset the model learns from
Would clean accuracy expose it?	No	No
Main defense focus	Harden inference or make the model more robust	Audit data and inspect learned representations
Examples from the labs	Adversarial training, feature squeezing	Activation clustering, spectral signatures
What the defender usually needs	Access to the model at evaluation/deployment time	Access to both the model and the training data

A simple way to remember this is:

A one-sentence takeaway

Lab 1 asks: "Can someone fool the model by changing the input now?"
Lab 2 asks: "Was the model taught a hidden bad rule during training?"

9.2 A practical decision guide for defense

Different situations call for different defenses. The table below is not a strict rulebook, but it gives a practical way to think about what to do first.

Situation	Recommended action
Training on untrusted or externally sourced data	Inspect the data pipeline before deployment. Use methods such as activation clustering or spectral signatures, and verify where the dataset came from.
Already-deployed model, no retraining possible	Use inference-time protections such as input preprocessing or feature squeezing as a first response. This may help against adversarial examples, but it will not remove a backdoor already learned during training.
Model received from a vendor as a black box	Treat the model as untrusted. Run behavioral tests, including trigger-style tests, and do not rely on clean accuracy alone.
High-stakes application (for example, medical AI)	Do not rely on a single defense. Combine data auditing, robustness testing, safer training procedures, and post-training behavioral checks. In other words: build defense in depth.

The main message here is that **there is no universal one-click defense**. A defense that helps in Lab 1 may do almost nothing in Lab 2, because the threat enters the system at a different stage.

Exercise 7: Final Reflection Across Both Labs

1. A hospital security team has resources for only one major security effort this year: hardening the inference pipeline (in Adversarial Defenses labs) or auditing the training data pipeline (defenses in this lab). Which would you recommend first? What information about the hospital's data collection or vendor process would change your answer?
2. Return to your hypotheses from the opening Think box of this lab. How has your understanding changed? Write one paragraph explaining a backdoor attack to a hospital administrator who has never studied machine learning.
3. **Both labs describe empirical defenses**—defenses that work well in practice against known attacks, but do not come with a proof against every possible attack. In your own words, what would a *certified* defense need to guarantee in "Adversarial Attacks and Defenses labs" and in this lab? Why is that much harder than showing good experimental results?

10 Summary

This lab introduced the basic logic of backdoor attacks: the model behaves normally on clean inputs, but secretly follows a hidden trigger rule that was planted during training. The summary below collects the main ideas in one place.

Concept	What it does	Key insight
Backdoor attack	Corrupts training data so the model learns a hidden trigger rule	A model can look normal on clean data and still contain a malicious behavior
Trigger δ	Adds a repeated pattern that activates the backdoor	The trigger must be consistent enough for the model to learn and subtle enough to avoid easy detection
Attack success rate (ASR)	Measures how often triggered inputs are sent to the target class	CA alone is not enough; a backdoor can hide behind high clean accuracy
Activation clustering	Looks for separable groups in internal representation space	Clean and trigger-driven target predictions may travel through different internal pathways
Spectral signatures	Scores samples by how strongly they align with a suspicious direction	Very high-scoring samples are the most suspicious, even if the separation is not perfect

Taken together, the two labs show a broader lesson about trustworthy machine learning:

Final takeaway

A model can fail for at least two very different reasons:

- the **input at test time** was manipulated, or
- the **training process itself** was poisoned.

If we only test clean accuracy, we may miss both.

References

Chen, B., et al. (2019). Detecting backdoor attacks by activation clustering. *AAAI Workshop on AI Safety*. arXiv:1811.03728

Tran, B., Li, J., & Madry, A. (2018). Spectral signatures in backdoor attacks. *NeurIPS 2018*. arXiv:1811.00636

Gu, T., Dolan-Gavitt, B., & Garg, S. (2019). BadNets. *IEEE Access*. arXiv:1708.06733